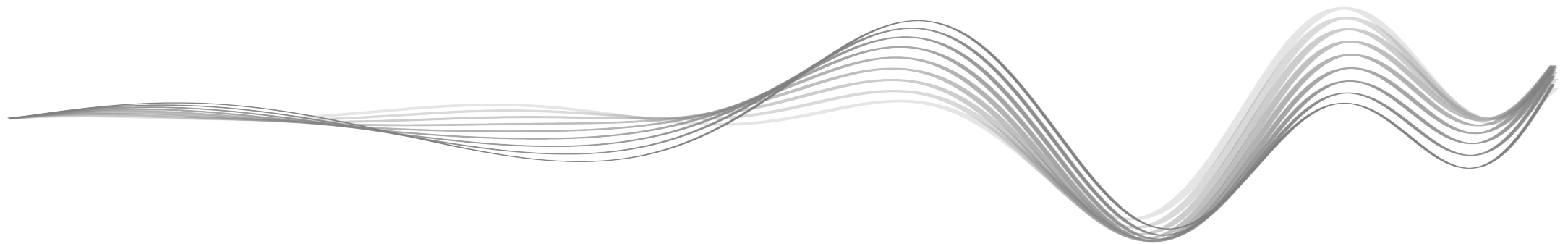




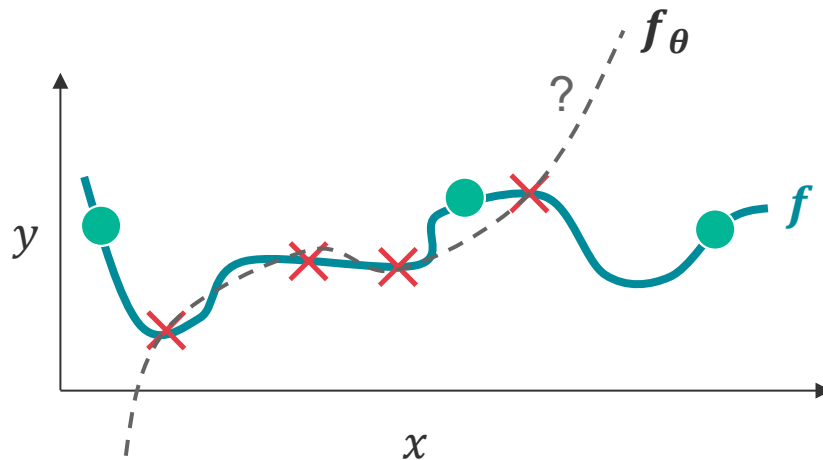
Deep Learning





Fundamentals

- **Universal method to approximate any complex function**, i.e. a mapping $y = f_{\theta}(x)$
 - x is a set of input quantities (independent variables)
 - y is a set of output quantities (dependent variables)
 - f is the actual relation between y and x , f_{θ} is the ANN approximation
- The mapping can be **high-dimensional in both x and y**

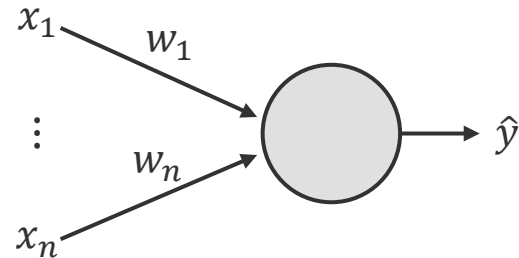


Approximate $f(x)$ from
sampling points $(x_i, y_i) \rightarrow$ **training data**

Accurate at
query points $(x, y)? \rightarrow$ **validation data**

True mapping function f
inaccessible, inexistent, unknown

Binary Perceptron

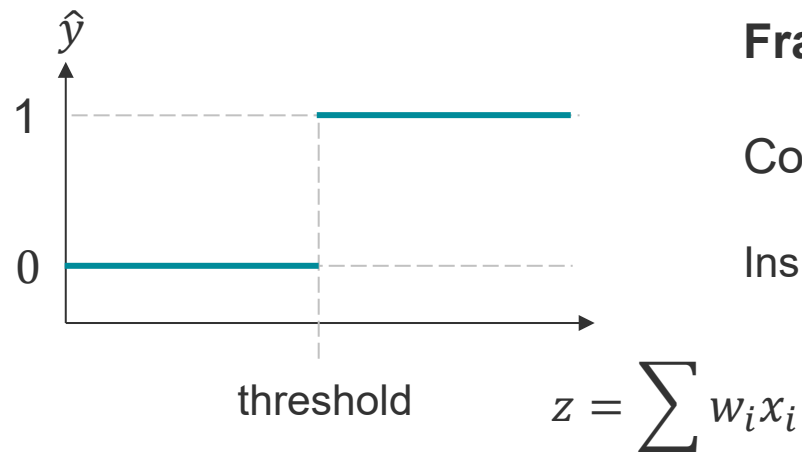


$$y \in \{0, 1\}, \quad w_i, x_i \in \mathbb{R}$$

$$\hat{y} = \begin{cases} 0 & \text{if } \sum w_i x_i \leq \text{threshold} \\ 1 & \text{if } \sum w_i x_i > \text{threshold} \end{cases}$$

„firing neuron“

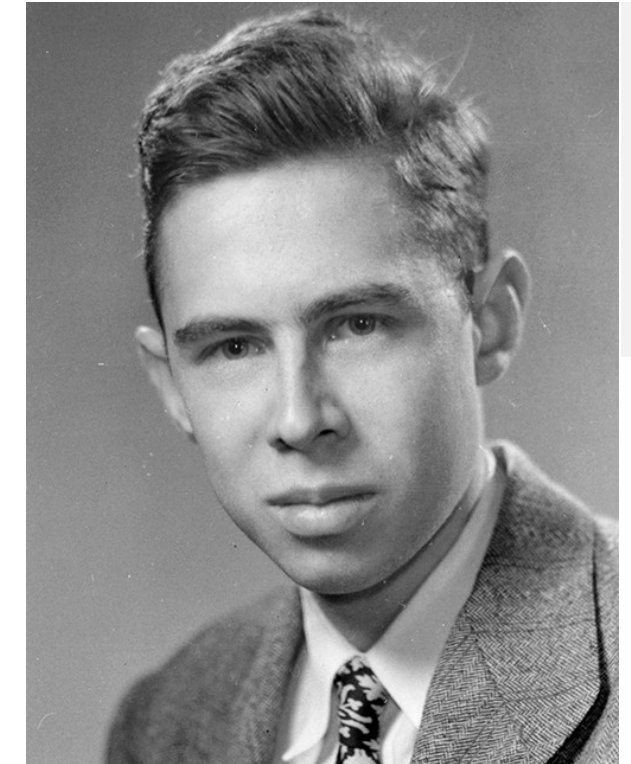
AND, OR, NAND gate



Frank Rosenblatt (1928 - 1971)

Considered one *father of ML*

Inspired work by W. McCulloch, W. Pitts

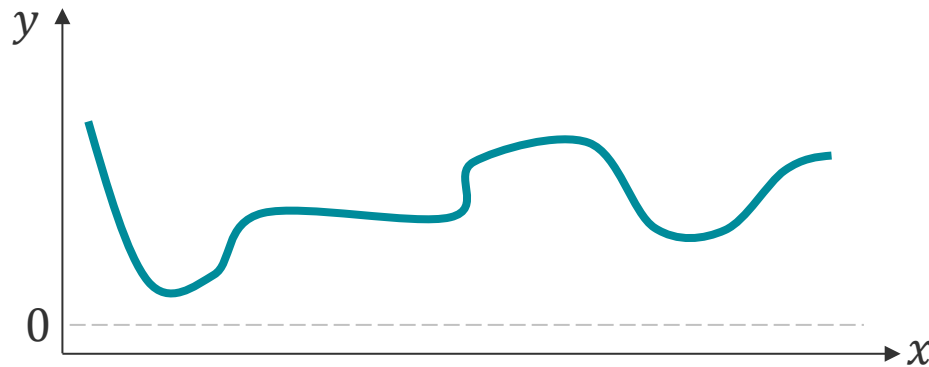


© Cornell University

Binary Perceptron

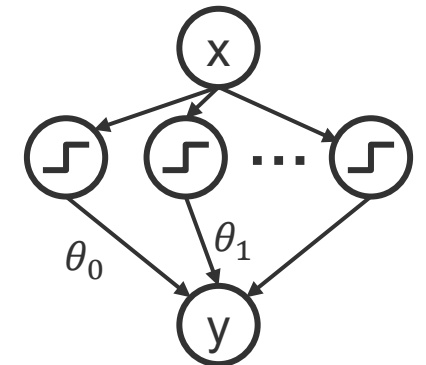
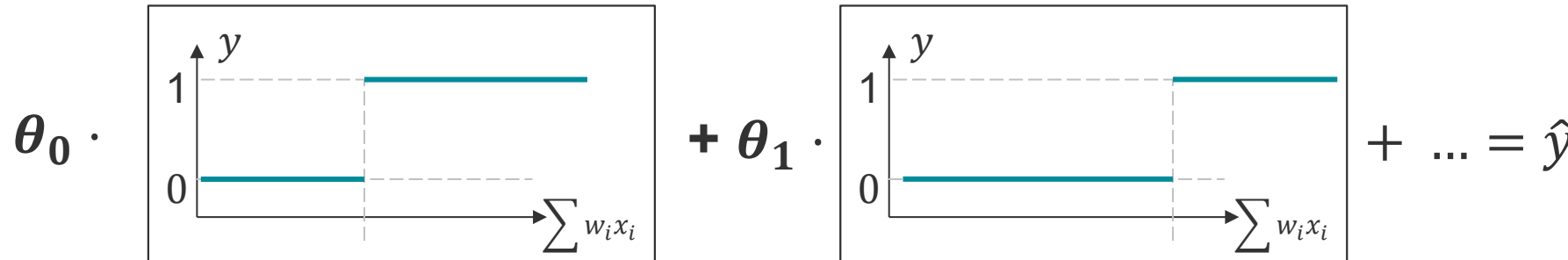


- Approximation of an unknown function using many binary perceptrons



Problem:

- $\hat{y} = f_{\theta}(x)$ not smooth nor continuous!
- Rapid output changes for small changes in the thresholds
- Very difficult to „train“

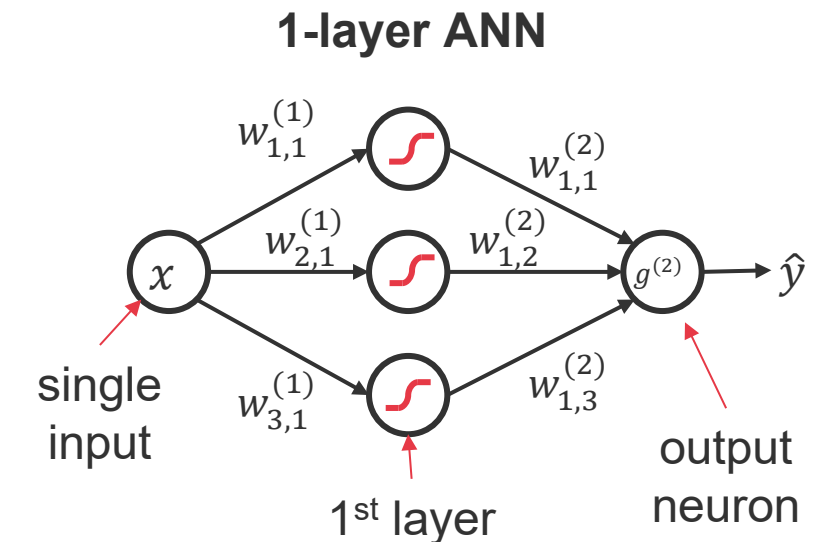
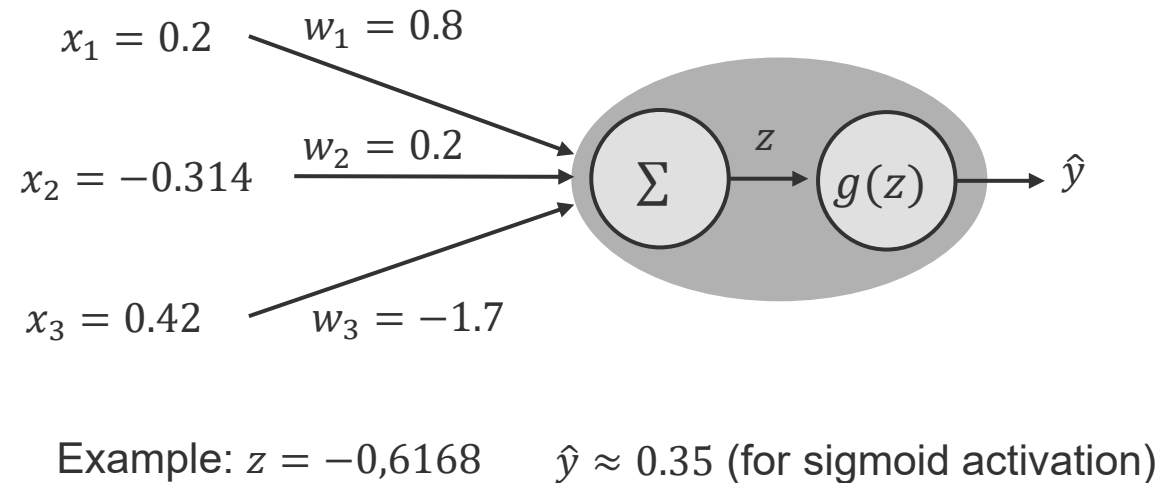
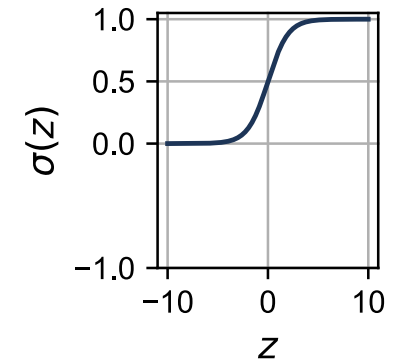


Neuron



- **Neuron** (*node, unit*): smallest element in an Artificial Neural Network (ANN)
- Neurons perform two tasks:
 - **Weighted summation** of inputs x_i : $z = \sum(w_i x_i)$
 - **(Non-) linear transformation: activation** $g(z)$

$$\text{sigmoid } \sigma(z) = \frac{1}{1+e^{-z}}$$
$$z \in \mathbb{R} \mapsto \sigma(z) \in [0,1]$$



Feed-forward Artificial Neural Network



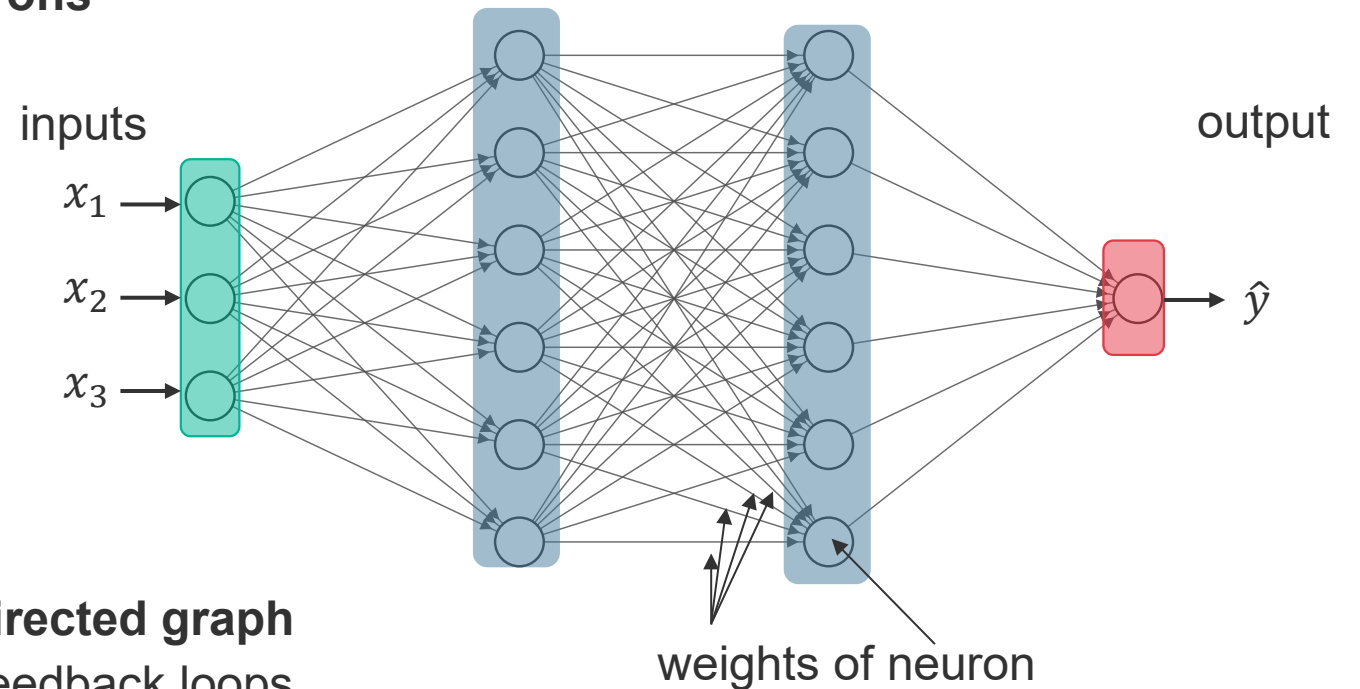
Cyber-Physical Systems
in Mechanical Engineering TU Berlin

- **Basic ANN:** feed-forward ANN (also called **multi-layered perceptron**, *MLP*, but built from neurons)

- **Layer-structured arrangement of neurons**

- **Input** layer
- Hidden layer(s)
- **Output** layer

- Notation for targets:
 y : ground truth values
 $\hat{y}$: predicted values



- **Fully connected feed-forward ANN: directed graph**

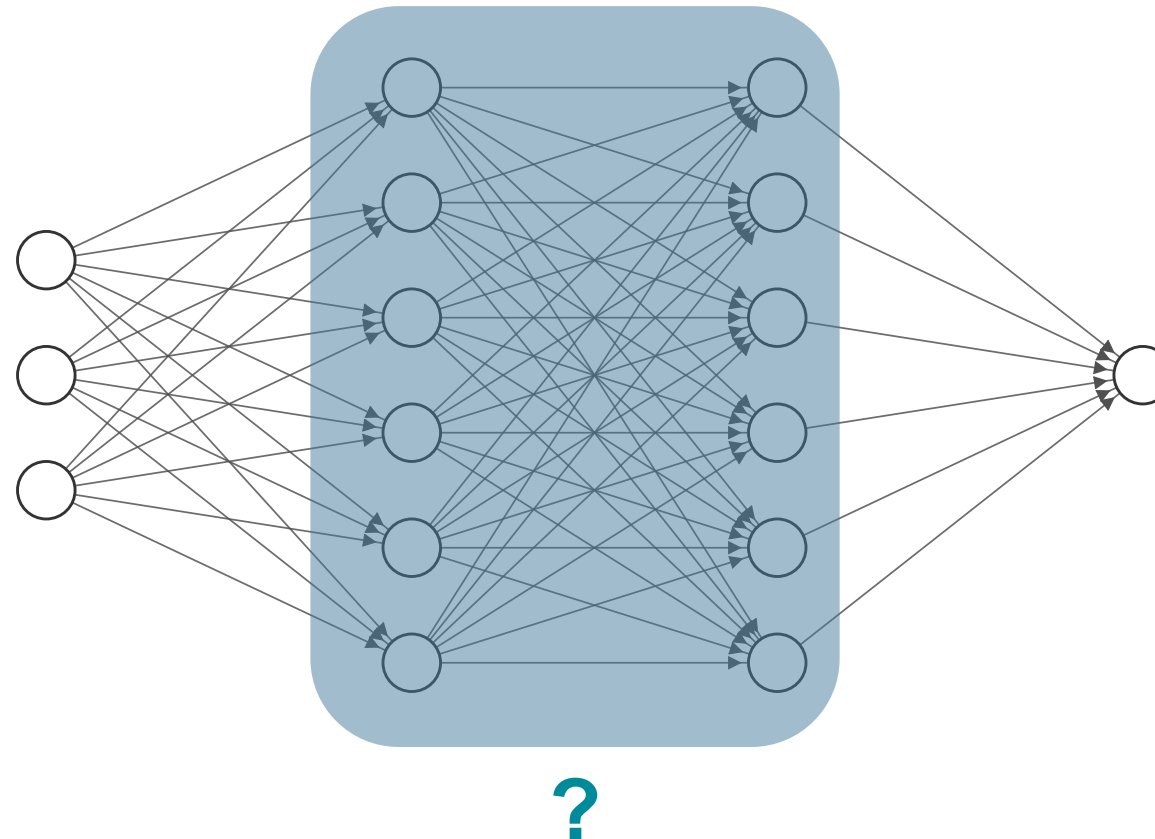
- Information flows only forward, no feedback loops
- Every neuron is connected to every neuron in the subsequent layer

Feed-forward Artificial Neural Network



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

- Architectural considerations
- **Prescribed** by learning task:
 - Input size (vector/image/...)
 - Output size (binary, ...)
- **Choice**/optimization:
 - No. of hidden layers
 - No. of hidden neurons
 - Activation, etc.





Forward Pass

Feeding Data Through Layers



- Data point $\mathbf{x} = [x_1, \dots, x_n]$ traverses the network in the forward direction to create a prediction $\hat{y}$

- Per layer** (here $l = 2$) $\mathbf{x}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)})$

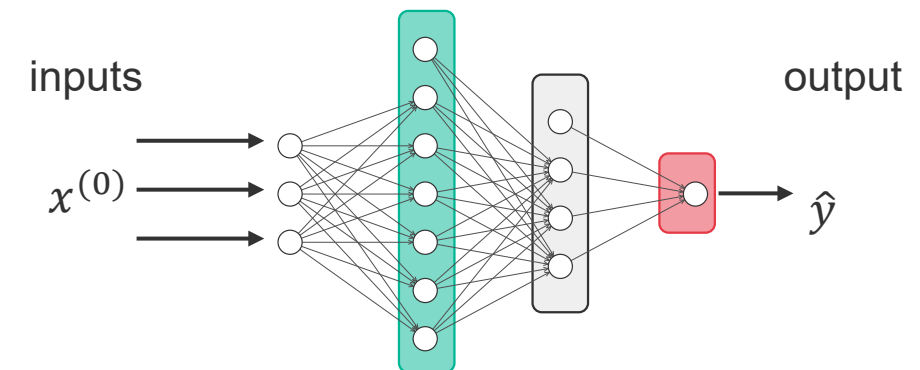
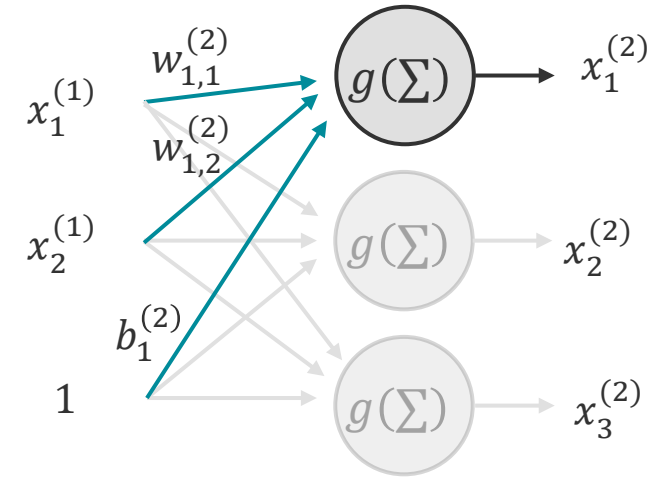
$$\begin{bmatrix} x_1^{(2)} \\ x_2^{(2)} \\ x_3^{(2)} \end{bmatrix} = g \left(\begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} \\ w_{3,1}^{(2)} & w_{3,2}^{(2)} \end{bmatrix} \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \end{bmatrix} + \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \\ b_3^{(2)} \end{bmatrix} \right)$$

row j : all incoming connections to neuron j in layer (l)

$\mathbf{W}^{(2)}$ of shape $(M^{(l)}, M^{(l-1)}) = (3, 2)$

- Complete network: **nested function** $x \mapsto \hat{y}$

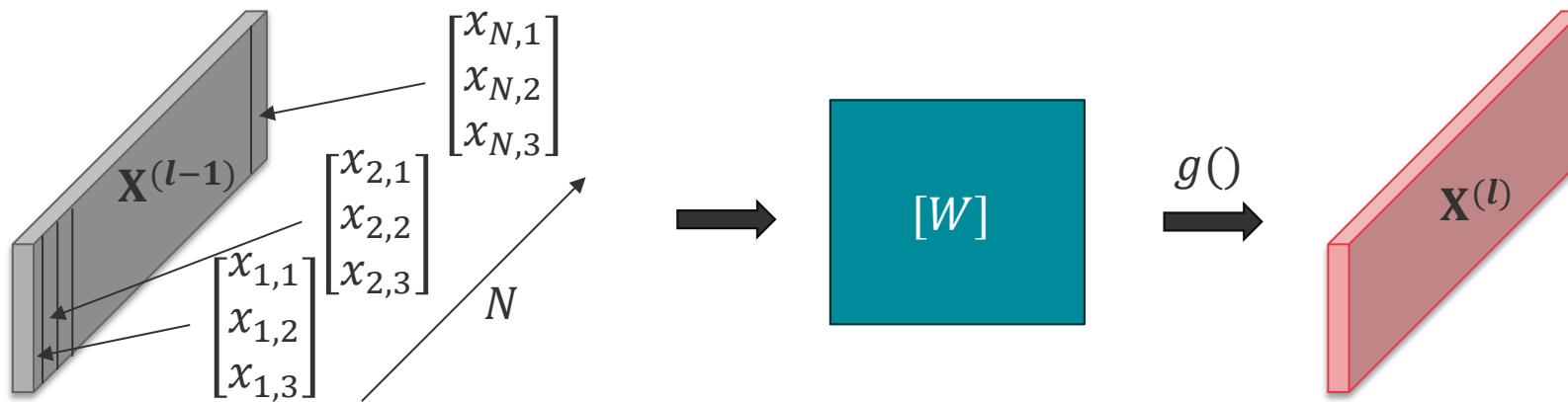
$$\hat{y} = g \left(\sum (w_i^{(3)}) g \left(\sum (w_i^{(2)}) g \left(\sum (w_i^{(1)}) x_i^{(0)} \right) \right) + b^{(2)} \right) + b^{(3)}$$



Tensors



- Previously: mathematics of a single data sample traversing the network
- In fact: one can **feed multiple samples at once** through the network

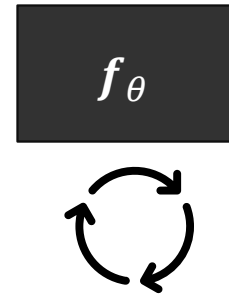


- Hardware acceleration: **GPUs** (many simple and independent computations)

Terms



- *Model training*: finding optimal weights and biases



- *Model inference*: feeding new inputs through the trained network to make a prediction





Activation Functions

- Activation functions $g(z)$ determine whether, or how much, a neuron is **activated**

$$x^{(l)} = g \left(\sum w_i^{(l)} x_i^{(l-1)} + b_i^{(l)} \right)$$

- Activation functions introduce nonlinearity into neural networks
- Activation functions are (mostly) saturating: activation level is bounded. Exception: ReLU
- Activation functions control the growth of weights and the behavior of gradients

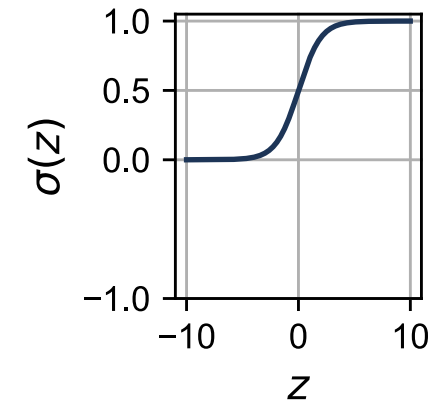
Sigmoid Activation Function



Sigmoid (logistic) function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\frac{\partial \sigma}{\partial z} = \sigma(z) \cdot (1 - \sigma(z))$$



- Behavior:
 - Maps $\mathbb{R}$ to $[0, 1]$, centered at 0, $\sigma(z = 0) = 0.5$
 - $\lim_{z \rightarrow \infty} \sigma(z) = 1$, $\lim_{z \rightarrow -\infty} \sigma(z) = 0$
 - Approximately: data (information) flow for $z > -5$, data (information) blocking for $z \ll -5$
- Usage:
 - Activation function in **hidden layers**
 - Activation function in **output layer** to **predict probabilities**
 - Most typical use case: **binary classification for a binomial probability distribution**

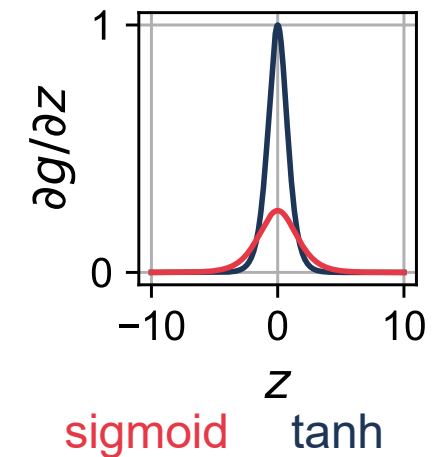
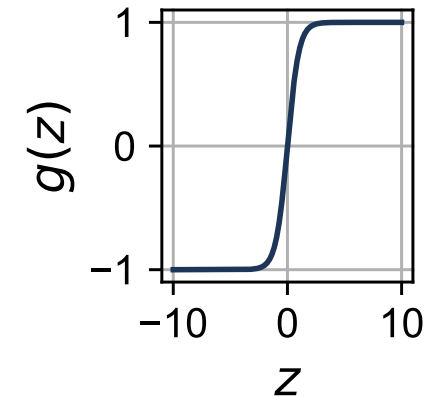
tanh Activation Function



Tangent hyperbolic function

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$
$$\frac{\partial g(z)}{\partial z} = 1 - g(z)^2$$

- Behavior:
 - Maps $\mathbb{R}$ to $[-1, 1]$, centered at 0, $g(z = 0) = 0.0$
 - $\lim_{x \rightarrow \infty} \sigma(z) = 1$, $\lim_{x \rightarrow -\infty} \sigma(z) = -1$
- Usage:
 - Activation function in **hidden layers**
 - Often better performance than sigmoid due to 4x larger gradient and symmetry in activation



ReLU Activation



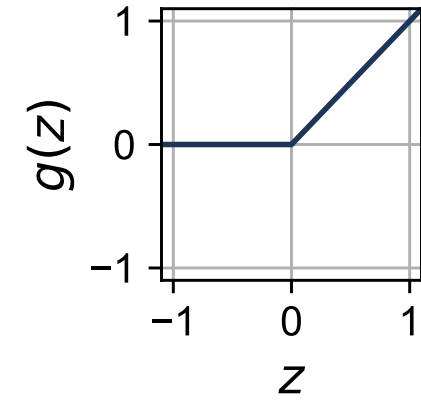
ReLU (Rectified linear unit)

$$g(z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} = \max(0, x)$$

$$\frac{\partial g(z)}{\partial z} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

- Behavior:
 - Maps $\mathbb{R}$ to $(0, \infty]$
 - Sets all input arguments to zero

- Usage:
 - Activation function in **hidden layers**
 - Solves the vanishing gradient problem, faster to evaluate than sigmoid or tanh
 - Activation function in output layer if regression target must not take negative values (rare case)



Activation Functions: Overview



activation	hidden layer	output layer	Task
linear		x	regression
tanh	x		classification & regression
sigmoid	x		classification & regression
sigmoid		x	binary classification
softmax		x	multi-class classification
ReLU	x	rare cases	classification & regression
Leaky ReLU	x		classification & regression



Loss Functions

Loss Functions for Regression Problems



Regression task: target variables y are of numeric type (ratio, interval)

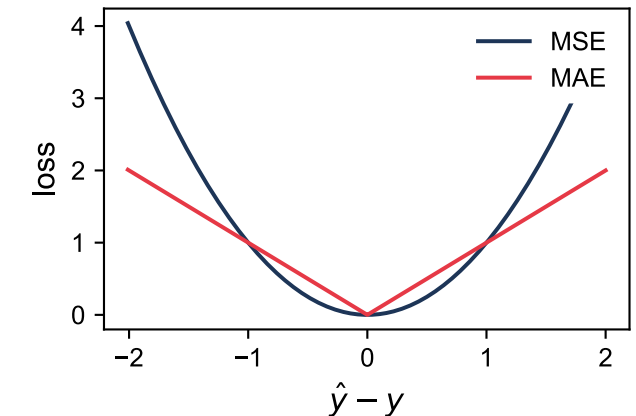
Classical loss functions: $\mathcal{L} = \|\mathbf{y} - \hat{\mathbf{y}}\|$ for $\mathbf{y}_i \in \mathbb{R}^m$, N data samples $\{x_i, y_i\}$

- **Mean absolute error (L_1)** $\mathcal{L} = \frac{1}{N} \sum_{i=0}^N |\mathbf{y}_i - \hat{\mathbf{y}}_i|$

robust to outliers, interpretable (unit of target variable)

- **Mean squared error (L_2)** $\mathcal{L} = \frac{1}{N} \sum_{i=0}^N |\mathbf{y}_i - \hat{\mathbf{y}}_i|^2$

emphasis on large errors, differentiable, assumes Gaussian distribution



- **Huber loss**
$$\mathcal{L} = \begin{cases} \frac{1}{2} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 & \text{for } \|\mathbf{y}_i - \hat{\mathbf{y}}_i\| \leq \delta \\ \delta \left(\|\mathbf{y}_i - \hat{\mathbf{y}}_i\| - \frac{1}{2} \delta \right) & \text{otherwise} \end{cases}$$

Loss Functions: Classification



Classification task: target variables y are of categorical type (nominal, ordinal)

- **Crossentropy** (log-loss)

$$\mathcal{L} = -\frac{1}{N} \sum_{i=0}^N y_i \cdot \log_2(\hat{y}_i) + (1 - y_i) \cdot \log_2(1 - \hat{y}_i), \quad y, \hat{y} \in [0,1]$$

binary classification tasks

- **Categorical crossentropy**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=0}^N \sum_{j=0}^C y_{ij} \cdot \log_2(\hat{y}_{ij})$$

multi-class classification tasks

- **Kullback-Leibler divergence** (KL divergence): for approximating probability distributions
- **Hinge loss** (max-margin loss): used for support vector machines



Optimization

The training loop

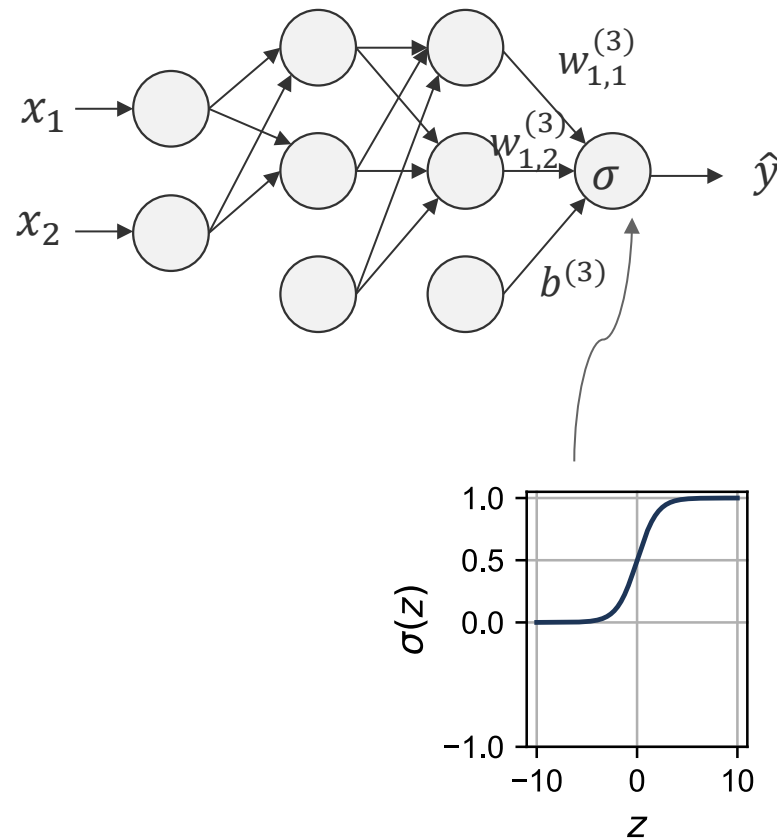


- **Model training** refers to
 - finding the optimal model parameters $\theta = [\mathbf{w}, \mathbf{b}]$
 - that minimize an error (the loss) of the predicted values $\hat{\mathbf{y}}_i$ w.r.t. to the desired outputs $\mathbf{y}_i$
 - using a set of ground truth input-output tuples $\{\mathbf{x}_i, \mathbf{y}_i\}, i = 1, \dots, N$

Training: Intuitive Approach



- How to change weights and biases?



Approach: find weights $\mathbf{W}$ and biases $\mathbf{b}$ such that the average loss $\mathcal{L}(\mathbf{x})$ (across all samples is) reduced.

1. Change **all** parameters ($\mathbf{w}_i$, $\mathbf{b}_i$) at once

2. Change each parameter based on **how** it affects the global loss value

- Increase in $w_i^{(l)}$ reduces $\mathcal{L} \rightarrow$ increase $w_i^{(l)}$
- Increase in $w_i^{(l)}$ increases $\mathcal{L} \rightarrow$ decrease $w_i^{(l)}$

3. Change each parameter based on **how much** it affects the global loss value

- Increase in $w_i^{(l)}$ reduces $\mathcal{L}$ **greatly**
- Increase in $w_i^{(l)}$ reduces $\mathcal{L}$ **only minimally**

Training = Search for Optimum



- How to get the directions that indicate **how (direction)** and **how much (amount)** each parameter needs to be changed?
- **Partial derivative of the loss value with respect to all model parameters $\theta = [\mathbf{w}, \mathbf{b}]$ for all N samples**

Error backpropagation

$$\nabla \mathcal{L}(\theta) = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \theta_1} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \theta_j} \end{bmatrix}$$

(direction of the steepest descent for all training samples)

largest absolute entry: largest effect on the loss function

$$\text{with } \frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \theta_1}(\mathbf{x}_i, \mathbf{y}_i)$$

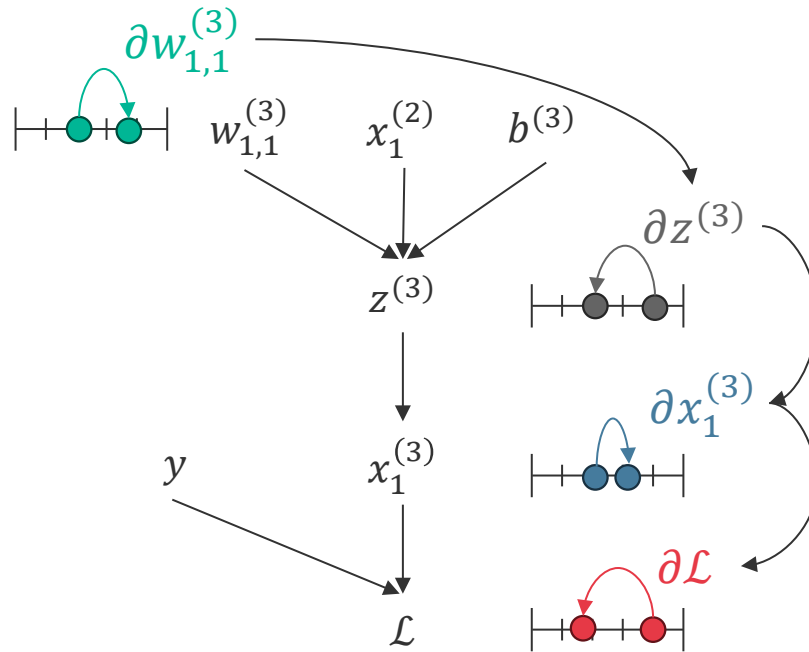
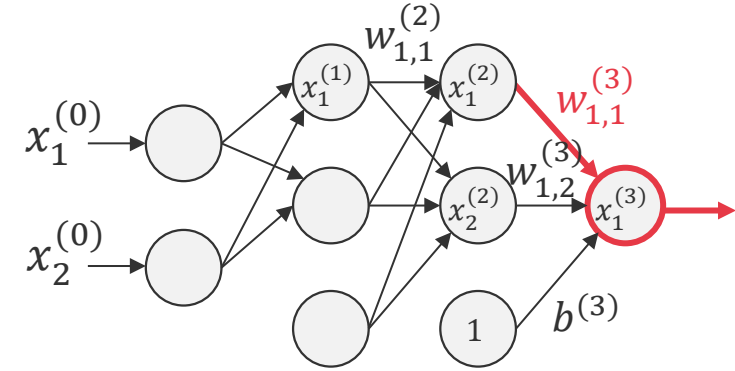
- Use a **gradient descent algorithm** to update all model parameters

$$\theta_{t+1} = \theta_t - \alpha \nabla \mathcal{L}(\theta)$$

Calculating Partial Derivatives



- Looking at a single weight and a single data sample
- Compute graph for the effect of $w_{1,1}^{(3)}$ on the loss $\mathcal{L}$:



$$\frac{\partial \mathcal{L}}{\partial w_{1,1}^{(3)}} = \frac{\partial z^{(3)}}{\partial w_{1,1}^{(3)}} \frac{\partial x_1^{(3)}}{\partial z^{(3)}} \frac{\partial \mathcal{L}}{\partial x_1^{(3)}}$$

change in $\mathcal{L}$
induced by w

change in z
induced by w

change in x
induced by z

change in $\mathcal{L}$
induced by x

$$z_j^{(l)} = \sum w_{j,i}^{(l)} x_i^{(l-1)} + b_j^{(l)}$$

$$x_j^{(l)} = \sigma(z_j^{(l)})$$

Inspired by 3Blue1Brown

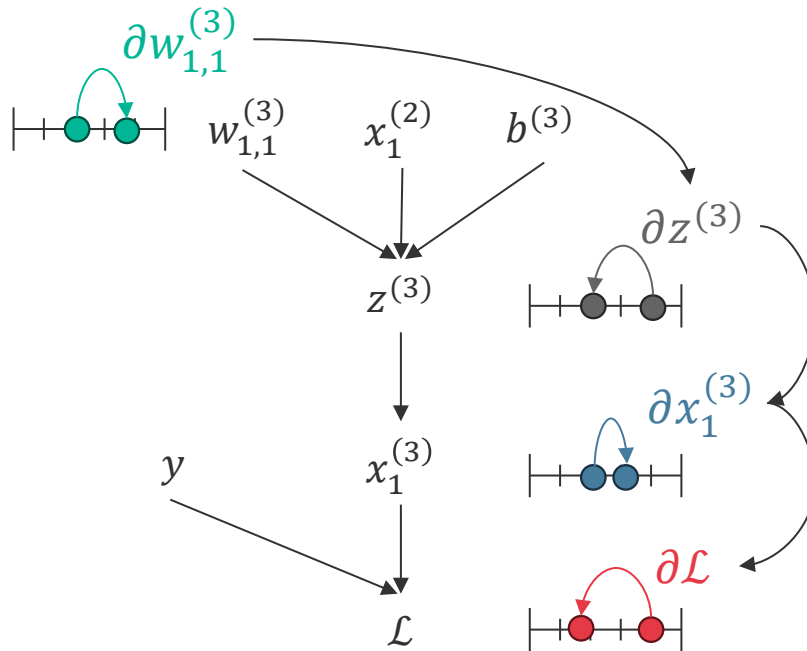
Calculating Partial Derivatives



- Looking at a single path and a single data sample
- Compute graph for the effect of $w_{1,1}^{(3)}$ on the loss $\mathcal{L}$:

$$z_j^{(l)} = \sum w_{j,i}^{(l)} x_i^{(l-1)} + b_j^{(l)}$$

$$x_j^{(l)} = \sigma(z_j^{(l)})$$



$$\frac{\partial \mathcal{L}}{\partial w_{1,1}^{(3)}} = \frac{\partial z^{(3)}}{\partial w_{1,1}^{(3)}} \frac{\partial x_1^{(3)}}{\partial z^{(3)}} \frac{\partial \mathcal{L}}{\partial x_1^{(3)}}$$

$$\frac{\partial z^{(3)}}{\partial w_{1,1}^{(3)}} = x_1^{(2)} = \frac{\partial}{\partial w_{1,1}^{(3)}} w_{1,1}^{(3)} x_1^{(2)} + b^{(3)}$$

$$\frac{\partial x_1^{(3)}}{\partial z^{(3)}} = \sigma(z^{(3)})(1 - \sigma(z^{(3)})) = \frac{\partial \sigma(z^{(3)})}{\partial z^{(3)}}$$

$$\frac{\partial \mathcal{L}}{\partial x_1^{(3)}} = 2(x_1^{(3)} - y) = \frac{\partial}{\partial x_1^{(3)}} (x_1^{(3)} - y)^2$$

Then:
plug in
values
from
forward
pass

Inspired by 3Blue1Brown

Wrap-Up: Training Neural Networks

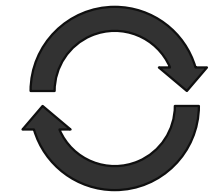


1. Forward pass

1. Feed all $i = 1, \dots, N$ training data samples through the network and obtain predictions $\hat{y}_i$
2. Compute error per predictions with respect to ground truth $\mathcal{L}(\mathbf{y}_i, \hat{y}_i)$

2. Backward pass

1. Propagate the error through the network by computing partial derivatives $\left. \frac{\partial \mathcal{L}}{\partial \theta_j} \right|_{x_i}$
2. Obtain vector of steepest descend $-\nabla \mathcal{L}(\boldsymbol{\theta})$



repeat

3. Optimize: Update model parameters $\boldsymbol{\theta}$ according to the gradient of the loss

Term: 1x (forward+backward+optimize pass of the complete training data set) = 1 „epoch“



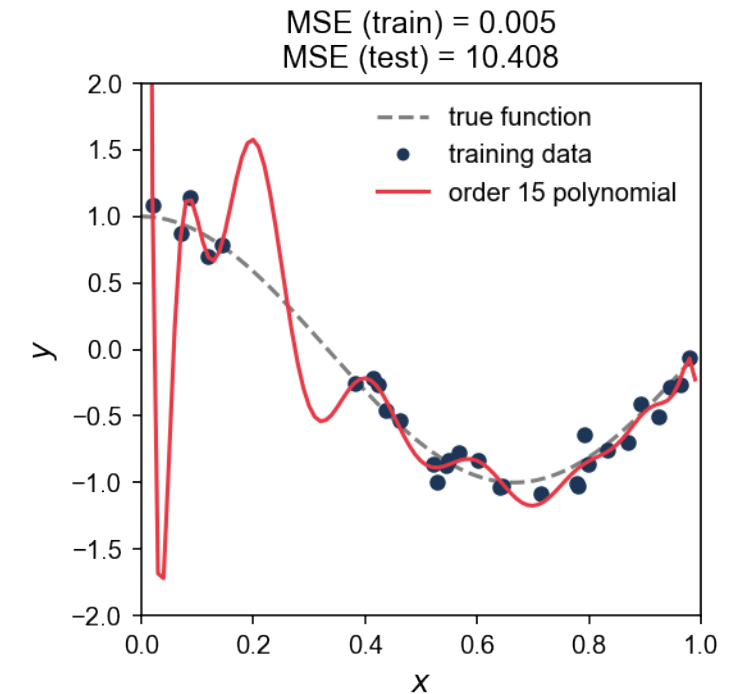
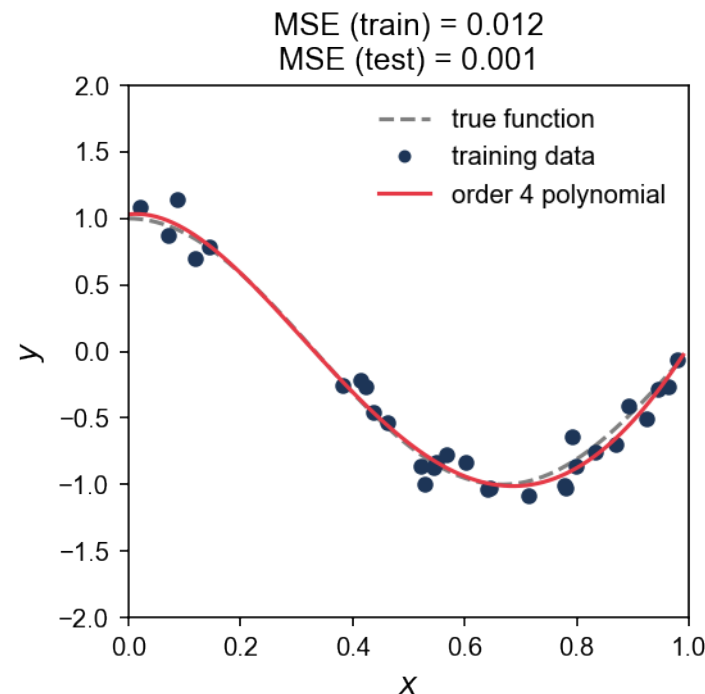
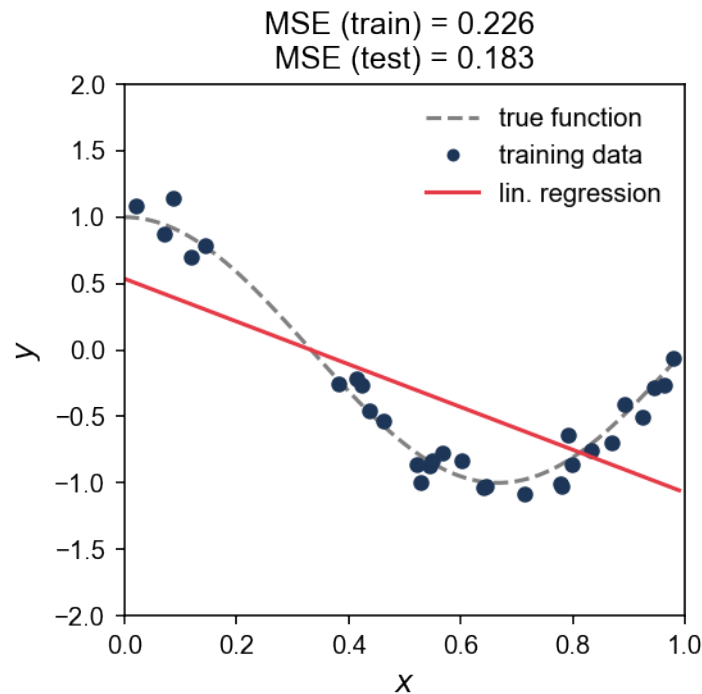
Generalization

Expected inference performance

Underfitting and Overfitting



- Fitting polynomials to (noisy) samples from $y = \cos(1.5\pi x)$

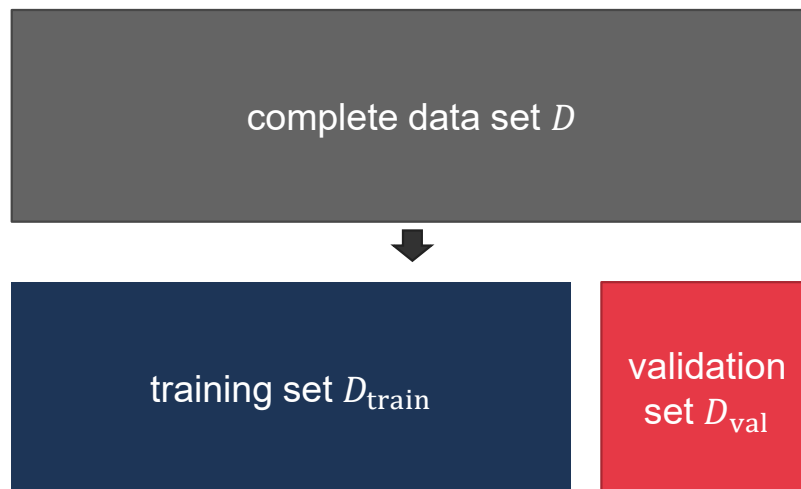


Example from [sklearn](https://scikit-learn.org/)

Neural Network Training (basic)



- **Aim:** find well-generalizing models with low bias and low variance
- **Generalization property:**
 - Generalization is the act of performing tasks of the same difficulty and nature
 - In data-driven modeling: predictive model performs well on previously unseen data



Typical procedure in machine learning:

1. Split data into training and test set
(typically at ratio of 0.7 / 0.3 or 0.8 / 0.2)
2. Find best model parameters θ on training set D_{train}
3. Evaluate performance on hold-out test set D_{test}

Neural Network Training



- Data set splitting
 - 0.7 - 0.15 - 0.15 (medium size)
 - 0.96 - 0.02 - 0.02 (big data)

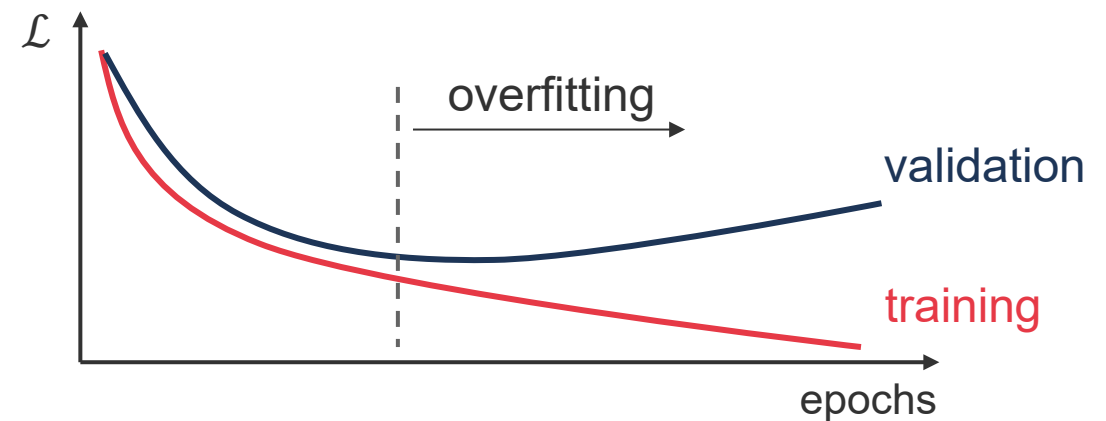


→ strict separation of model training and model evaluation!

- Model training
 - **Optimize** model parameters on **training** data set
 - **Track** of model performance on **validation** set *during training* → under / overfitting
 - **Report** performance of model on **test** data *after training* → generalization

- Stop model training when model starts to overfit the training data set. Early stopping

- **Attention:** ensure similar feature and label distributions in all sets before training!



Loss Function vs. Metric



Evaluate the model on a hold-out data set that was **not** used for training the model



1. model training

Find optimal model parameters θ for approximating $f(\theta): \mathbf{X} \mapsto \mathbf{y}$ using

the **training data set**

$$D_{\text{train}} = \{(\mathbf{x}_i, \mathbf{y}_i) \mid i = 1, \dots, N_{\text{train}}\}, \mathbf{x} \in \mathbb{R}^d$$

and a **loss function** $\mathcal{L}(\cdot)$, e.g. MSE

2. model evaluation

Evaluate the model's performance by making predictions $\hat{\mathbf{y}}_i = f_{\theta}(\mathbf{x}_i)$ on

the **test data set**

$$D_{\text{test}} = \{(\mathbf{x}_i, \mathbf{y}_i) \mid i = 1, \dots, N_{\text{test}}\}, \mathbf{x} \in \mathbb{R}^d$$

and evaluate a **metric** $\|\mathbf{y} - \hat{\mathbf{y}}\|$, e.g. R^2



Questions?